

CẨM NANG TOÀN DIỆN VỀ CÚ PHÁP & CƠ CHẾ C++ DÀNH CHO HỌC SINH CHUYÊN TIN & LẬP TRÌNH THI ĐẤU (CP)

Tác giả: **MINDA Learning Systems** | Dành cho HSG Tin Học 8, Chuyên Tin & Olympic

Contents

1	Tối Ưu Hóa Nhập/Xuất & Cơ Chế Đệm (Fast I/O & Buffer Flush)	1
1.1	Khái Niệm Bộ Đệm (Buffer) & Lệnh Flush Là Gì?	2
1.2	Tại Sao <code>std::endl</code> Lại Gây Ra Lỗi TLE?	2
1.3	Bộ 3 Lệnh Thần Thánh: <code>sync_with_stdio</code> & <code>cin.tie</code>	2
1.4	Xử Lý Đọc/Ghi File Chuẩn Đề Thi (.INP / .OUT)	2
2	Kiểu Dữ Liệu, Tràn Số & Khái Niệm Sai Số Epsilon (ϵ)	3
2.1	Các Kiểu Số Nguyên & Giới Hạn Bộ Nhớ	3
2.2	Số Thực Chuẩn IEEE 754 & Khái Niệm Sai Số Epsilon (ϵ)	3
2.2.1	Quy Tắc So Sánh Số Thực Bằng Epsilon (ϵ)	4
2.2.2	Định Dạng In Số Thực Đúng Chuẩn	4
3	Đại Số Modulo & Phép Chia Trong Trường Thặng Dư $\mathbb{Z}_p$	4
3.1	Các Tính Chất Cơ Bản Của Phép Modulo	4
3.2	Phép Chia Modulo & Nghịch Đảo Nhân (Modular Inverse)	4
3.3	Cài Đặt Bộ Hàm Toán Modulo Chuẩn Mực	5
4	Phân Bố Bộ Nhớ (Stack vs Data Segment) & Mẹo Tối Ưu RAM	5
4.1	Kỹ Thuật Padding Mảng & Cấu Trúc <code>std::bitset</code>	6
5	Các Hàm Tích Hợp Phần Cứng & STL Tối Ưu Tốc Độ	6
5.1	Hàm Thao Tác Bit Tối Ưu Bằng Chỉ Thị Phần Cứng (GCC Built-ins)	6
5.2	Tìm Kiếm Nhị Phân Trong Mảng Đã Sắp Xếp	6
6	Khung Mã Nguồn C++ Chuẩn Dành Cho Thi Đấu	7

1 Tối Ưu Hóa Nhập/Xuất & Cơ Chế Đệm (Fast I/O & Buffer Flush)

Trong các kỳ thi lập trình (HSG, VNOI, VOI, Codeforces), khối lượng dữ liệu đầu vào và đầu ra thường lên đến $10^5 - 10^6$ số hoặc dòng. Việc không hiểu rõ cơ chế đệm (I/O Buffer) là nguyên nhân hàng đầu khiến thuật toán có độ phức tạp đúng nhưng vẫn bị **Time Limit Exceeded (TLE)**.

1.1 Khái Niệm Bộ Đệm (Buffer) & Lệnh Flush Là Gì?

Cơ chế hoạt động của I/O Buffer trong Hệ Điều Hành

- **I/O Buffer (Bộ nhớ đệm):** Nhập/xuất trực tiếp ra màn hình console hoặc file đĩa cứng là một thao tác phần cứng rất chậm (tốn hàng ngàn chu kỳ CPU vì cần chuyển quyền kiểm soát từ *User Space* sang *Kernel Space* thông qua *System Call* `write()`).
- Để tối ưu, thư viện C++ gom các dữ liệu cần in vào một vùng nhớ tạm trên RAM gọi là **Output Buffer** (thường có kích thước 4KB hoặc 8KB). Khi buffer này bị đầy, hệ điều hành mới xả toàn bộ dữ liệu ra màn hình/file một lần duy nhất.
- **Flush Buffer:** Là hành động **ép buộc** hệ điều hành phải đẩy dữ liệu hiện có trong buffer ra đĩa/màn hình ngay lập tức, bất kể buffer đã đầy hay chưa.

1.2 Tại Sao `std::endl` Lại Gây Ra Lỗi TLE?

Rất nhiều bạn có thói quen viết: `cout << ans << endl;`. Bản chất trong thư viện C++:

`std::endl ≡ '\n' + std::flush;`

Nếu bài toán yêu cầu in ra 10^5 dòng kết quả:

- Dùng `'\n'`: Dữ liệu được lưu trong RAM buffer, chỉ xả đĩa khoảng $\approx 20 - 30$ lần. Thời gian thực thi: ≈ 0.02 giây.
- Dùng `endl`: Bạn kích hoạt 100.000 lần **system call flush()**. CPU liên tục bị ngắt quãng. Thời gian thực thi tăng vọt lên: $\approx 1.5 - 2.5$ giây → **Bị TLE oan uổng!**

1.3 Bộ 3 Lệnh Thần Thánh: `sync_with_stdio` & `cin.tie`

```
1 void setupFastIO() {
2     // 1. Ngắt dòng bo giữa C++ stream (cin/cout) và C stream (scanf/printf)
3     ios_base::sync_with_stdio(false);
4
5     // 2. Huy liên kết tự động giữa cin và cout
6     cin.tie(NULL);
7     cout.tie(NULL);
8 }
```

- **`sync_with_stdio(false)`:** Mặc định C++ đồng bộ luồng `cin/cout` với `scanf/printf` để lập trình viên có thể dùng lẫn lộn cả hai. Việc duy trì tính đồng bộ này tốn rất nhiều thời gian quản lý con trỏ đệm. Khi tắt đi, tốc độ `cin/cout` sẽ nhanh ngang ngửa hoặc vượt `scanf/printf`. (Lưu ý: Sau khi tắt, không được dùng lẫn `printf/scanf` trong cùng chương trình).
- **`cin.tie(NULL)`:** Mặc định, mỗi khi gọi `cin`, C++ sẽ tự động flush `cout` để đảm bảo màn hình đã in hết các dòng nhắc lệnh (prompt). Trong lập trình thi đấu, máy chấm chỉ so khớp output cuối cùng nên việc flush này là hoàn toàn dư thừa.

1.4 Xử Lý Đọc/Ghi File Chuẩn Đề Thi (.INP / .OUT)

```

1 #define TASK "BAI1"
2
3 void setupFile() {
4     #ifndef ONLINE_JUDGE
5         if (fopen(TASK ".INP", "r")) {
6             freopen(TASK ".INP", "r", stdin);
7             freopen(TASK ".OUT", "w", stdout);
8         }
9     #endif
10 }

```

Đoạn mã trên tự động kiểm tra nếu có file BAI1.INP trong thư mục thì chuyển hướng luồng đọc từ file, còn khi nộp lên các trang Online Judge (như Codeforces/VNOI) thì tự động dùng bàn phím chuẩn stdin/stdout.

2 Kiểu Dữ Liệu, Tràn Số & Khái Niệm Sai Số Epsilon (ϵ)

2.1 Các Kiểu Số Nguyên & Giới Hạn Bộ Nhớ

Kiểu dữ liệu	Kích thước	Miền giá trị (Xấp xỉ)	Ứng dụng trong thi đấu
int	32-bit (4 bytes)	$[-2.14 \times 10^9, 2.14 \times 10^9]$	Đếm, chỉ số mảng, giá trị $\leq 10^9$
long long	64-bit (8 bytes)	$[-9.22 \times 10^{18}, 9.22 \times 10^{18}]$	Tính tổng, tích, khoảng cách, quy hoạch động
__int128_t	128-bit (16 bytes)	$\approx [-1.7 \times 10^{38}, 1.7 \times 10^{38}]$	Nhân 2 số 64-bit tránh tràn trước khi modul

Lỗi Tràn Số Trung Gian (Implicit Intermediate Overflow)

Xét đoạn mã sau:

```

1 int a = 1000000;
2 int b = 1000000;
3 long long c = a * b; // SAI! a * b tính theo kiểu int (bị tràn số trước khi gán vào c)

```

Cách sửa chuẩn: Bắt buộc ép kiểu toán hạng sang 64-bit bằng hằng số 1LL:

```

1 long long c = 1LL * a * b; // DUNG! 1LL ép phép nhân tính toán ở kiểu long long

```

2.2 Số Thực Chuẩn IEEE 754 & Khái Niệm Sai Số Epsilon (ϵ)

Bản chất số thực trong máy tính & Tại sao $0.1 + 0.2 \neq 0.3$?

- Máy tính lưu trữ số thực bằng hệ nhị phân (chuẩn IEEE 754). Nhiều phân số thập phân quen thuộc (như $0.1 = \frac{1}{10}$) khi đổi sang hệ nhị phân sẽ trở thành **số tuần hoàn vô hạn**:

$$0.1_{10} = 0.0001100110011..._2$$

- Do số bit lưu trữ có hạn (53 bit phần định trị với double), số bị cắt cụt dẫn đến sai số làm tròn (*round-off error*).
- Vì vậy, trong C++, biểu thức $(0.1 + 0.2 == 0.3)$ sẽ trả về **false**!

2.2.1 Quy Tắc So Sánh Số Thực Bằng Epsilon (ϵ)

Epsilon (ϵ) là một hằng số cực nhỏ (thường chọn $\epsilon = 10^{-9}$), biểu thị ngưỡng sai số có thể chấp nhận được.

```
1 const double EPS = 1e-9;
2
3 // 1. Kiểm tra a bằng b (a == b)
4 bool isEqual(double a, double b) {
5     return abs(a - b) < EPS;
6 }
7
8 // 2. Kiểm tra a lớn hơn b (a > b)
9 bool isGreater(double a, double b) {
10    return a > b + EPS;
11 }
12
13 // 3. Kiểm tra a nhỏ hơn hoặc bằng b (a <= b)
14 bool isLessOrEqual(double a, double b) {
15    return a < b + EPS;
16 }
```

2.2.2 Định Dạng In Số Thực Đúng Chuẩn

Để in số thực với đúng k chữ số sau dấu phẩy (không bị hiển thị dạng khoa học $1.23e + 06$):

```
1 #include <iomanip>
2 cout << fixed << setprecision(2) << ans << '\n'; // In 2 chu so thập phân
```

3 Đại Số Modulo & Phép Chia Trong Trường Thặng Dư $\mathbb{Z}_p$

Modulo ($a \pmod m$) là phép toán lấy phần dư của phép chia nguyên a cho m . Hầu hết các bài toán đếm tổ hợp yêu cầu kết quả theo modulo $10^9 + 7$ hoặc 998244353.

3.1 Các Tính Chất Cơ Bản Của Phép Modulo

1. $(a + b) \pmod m = ((a \pmod m) + (b \pmod m)) \pmod m$
2. $(a - b) \pmod m = ((a \pmod m) - (b \pmod m) + m) \pmod m$ (Cộng thêm m để chống kết quả âm trong C++)
3. $(a \times b) \pmod m = ((a \pmod m) \times (b \pmod m)) \pmod m$
4. **CẢNH BÁO:** Phép chia **KHÔNG** có tính chất phân phối: $(a/b) \pmod m \neq ((a \pmod m)/(b \pmod m)) \pmod m$.

3.2 Phép Chia Modulo & Nghịch Đảo Nhân (Modular Inverse)

Để tính $\frac{a}{b} \pmod m$, ta chuyển phép chia thành phép nhân với **nghịch đảo modulo**:

$$\frac{a}{b} \pmod m \iff a \times b^{-1} \pmod m$$

Định Lý Nhỏ Fermat (Fermat's Little Theorem)

Nếu m là một số nguyên tố và b không chia hết cho m , thì:

$$b^{m-1} \equiv 1 \pmod{m} \implies b \times b^{m-2} \equiv 1 \pmod{m}$$

Do đó, nghịch đảo modulo của b theo modulo nguyên tố m chính là:

$$b^{-1} \equiv b^{m-2} \pmod{m}$$

3.3 Cài Đặt Bộ Hàm Toán Modulo Chuẩn Mực

```

1  const int MOD = 1e9 + 7;
2
3  // Cong modulo
4  int add(int a, int b) {
5      return (a + b >= MOD) ? (a + b - MOD) : (a + b);
6  }
7
8  // Tru modulo (chong so am)
9  int sub(int a, int b) {
10     return (a - b < 0) ? (a - b + MOD) : (a - b);
11 }
12
13 // Nhan modulo (chong tran 64-bit)
14 int mul(int a, int b) {
15     return (1LL * a * b) % MOD;
16 }
17
18 // Luy thua nhi phan nhanh a^b % MOD trong O(log b)
19 long long power(long long a, long long b) {
20     long long res = 1;
21     a %= MOD;
22     while (b > 0) {
23         if (b & 1) res = (1LL * res * a) % MOD;
24         a = (1LL * a * a) % MOD;
25         b >>= 1;
26     }
27     return res;
28 }
29
30 // Chia modulo: a / b % MOD
31 long long divMod(long long a, long long b) {
32     return mul(a, power(b, MOD - 2));
33 }

```

4 Phân Bố Bộ Nhớ (Stack vs Data Segment) & Mẹo Tối Ưu RAM**Lỗi Tràn Ngăn Xếp (Stack Overflow / SIGSEGV)**

Nếu khai báo mảng 10^6 phần tử bên trong hàm `main()`:

```

1  int main() {
2      int a[1000000]; // 4MB - ấRt ể làm tràn Stack ếu có thêm đệ quy -> CRASH ngay
        khi ạy!
3  }

```

Bố Cục Bộ Nhớ Của Tiến Trình C++ (Process Memory Layout)

- **Stack Segment:** Lưu trữ các biến cục bộ trong hàm, tham số truyền vào hàm và lời gọi đệ quy. Dung lượng mặc định rất nhỏ (thường chỉ **8MB**).
- **BSS / Data Segment (Bộ nhớ toàn cục):** Lưu trữ các biến và mảng toàn cục (*Global Scope*). Dung lượng có thể chiếm trọn bộ nhớ RAM được cấp cho bài thi (ví dụ **256MB** - **512MB**).
- **Heap Segment:** Cấp phát động (`new`, `malloc`, `std::vector`).

Quy tắc vàng: Mọi mảng có kích thước $> 10^5$ **BẮT BUỘC** phải khai báo toàn cục (*Global Scope*) bên ngoài hàm `main()`!

4.1 Kỹ Thuật Padding Mảng & Cấu Trúc `std::bitset`

1. **Padding mảng ($MAXN + 7$):** Khi làm việc với mảng 1-indexed (từ 1 đến N), luôn cộng thêm 5 đến 7 phần tử để tránh lỗi truy cập tràn biên (*Out of bounds*):

```
1  const int MAXN = 1e6 + 7;
2  int a[MAXN]; // Khai bao toan cuc, tu dong khoi tao gia tri 0
3
```

2. **`std::bitset` - Tiết kiệm $8 \times$ RAM:** Kiểu `bool` trong C++ chiếm 1 byte (8 bits) cho mỗi phần tử. Trong khi đó, `std::bitset` nén mỗi trạng thái boolean thành đúng 1 bit:

```
1  bitset<100000000> is_prime; // Sang nguyen to den 10^8 chi ton ~12 MB RAM thay vi 100 MB!
2
```

5 Các Hàm Tích Hợp Phần Cứng & STL Tối Ưu Tốc Độ

5.1 Hàm Thao Tác Bit Tối Ưu Bằng Chỉ Thị Phần Cứng (GCC Built-ins)

Các hàm này được trình biên dịch dịch trực tiếp thành **1 lệnh CPU duy nhất** ($\mathcal{O}(1)$):

Hàm C++	Ý nghĩa	Ví dụ
<code>__builtin_popcount(x)</code>	Đếm số lượng bit 1 của số 32-bit	<code>__builtin_popcount(13) = 3</code>
<code>__builtin_popcountll(x)</code>	Đếm số lượng bit 1 của số 64-bit	Dùng cho biến <code>long long</code>
<code>__builtin_clz(x)</code>	Đếm số bit 0 liên tiếp ở đầu (<i>leading zeros</i>)	Dùng tính $\lfloor \log_2(x) \rfloor = 31 - \text{__builtin_clz}(x)$
<code>__builtin_ctz(x)</code>	Đếm số bit 0 liên tiếp ở cuối (<i>trailing zeros</i>)	Dùng tìm vị trí bit 1 nhỏ nhất

5.2 Tìm Kiếm Nhị Phân Trong Mảng Đã Sắp Xếp

- `lower_bound(a, a + n, x)`: Trả về con trỏ/iterator tới phần tử đầu tiên $\geq x$.

- `upper_bound(a, a + n, x)`: Trả về con trỏ/iterator tới phần tử đầu tiên $> x$.
- Số lượng phần tử có giá trị bằng x trong đoạn: `upper_bound(...)` - `lower_bound(...)`.

6 Khung Mã Nguồn C++ Chuẩn Dành Cho Thi Đấu

```

1  /**
2   * Author: MINDA Competitive Programming Handbook
3   * Task:   Standard Multi-Purpose CP Template
4   */
5  #include <bits/stdc++.h>
6  using namespace std;
7
8  // — MACROS & SHORTHANDS —
9  #define TASK          "PROBLEM"
10 #define all(x)         (x).begin(), (x).end()
11 #define sz(x)          (int)((x).size())
12 #define fi             first
13 #define se             second
14 #define pb             push_back
15 #define FOR(i,a,b)     for (int i = (a); i <= (b); ++i)
16 #define FOD(i,a,b)     for (int i = (a); i >= (b); --i)
17
18 typedef long long ll;
19 typedef pair<int, int> pii;
20 typedef vector<int> vi;
21
22 const int INF = 1e9 + 7;
23 const ll LINF = 1e18 + 7;
24 const int MOD = 1e9 + 7;
25 const double EPS = 1e-9;
26
27 // — FAST I/O & FILE REDIRECTION —
28 void setupIO() {
29     ios_base::sync_with_stdio(false);
30     cin.tie(NULL);
31     cout.tie(NULL);
32
33     #ifndef ONLINE_JUDGE
34         if (fopen(TASK ".INP", "r")) {
35             freopen(TASK ".INP", "r", stdin);
36             freopen(TASK ".OUT", "w", stdout);
37         }
38     #endif
39 }
40
41 // — MAIN ALGORITHM SOLVER —
42 void solve() {
43     // Viết thuật toán giải quyết tại đây
44 }
45
46 int main() {
47     setupIO();
48
49     int testCount = 1;
50     // cin >> testCount; // Mo comment neu bai toan co nhieu test cases
51     while (testCount--) {
52         solve();
53     }
54 }

```

```
55  
56     return 0;  
57 }
```

Listing 1: Competitive Programming Master Template